PCA

Jyotishka Datta (jyotishka@vt.edu)
Department of Statistics, Virginia Tech

Source

- A lot of R codes in this demo are taken from R codes in the excellent book “Introduction to Statistical Learning with Applications in R” by James, Witten, Hastie and Tibshirani. The Book website has R codes, labs and lots of other resources http://www.statlearning.com/.
- The Crime data and Iris data codes were provided by Dr. Sumanta Basu. PCR codes & examples were provided by Dr. Bobby Gramacy.

Recap (PCA)

- In practice, we observe n observations of the $(x_1, \ldots, x_p)$.
- Let X be a data matrix with n rows and p columns. Subtract the column mean from each variable.
- The first principal component direction is the vector $\beta_1 = (\beta_{11}, \ldots, \beta_{p1})'$ s.t.

$$\max_{\beta_{11},\ldots,\beta_{p1}} \left\{ \frac{1}{n} \left(\sum_{j=1}^{p} \beta_{j1} x_{ij} \right)^2 \right\} \text{ subject to } \sum_{j=1}^{p} \beta_{j1}^2 = 1$$

Recap (PCA)

$$\max_{\beta_{11},\ldots,\beta_{p1}} \left\{ \frac{1}{n} \left(\sum_{j=1}^{p} \beta_{j1} x_{ij} \right)^2 \right\} \text{ subject to } \sum_{j=1}^{p} \beta_{j1}^2 = 1$$

- $\sum_{j=1}^{p} \beta_{j1} x_{ij}$: Projection of ith sample into β_1 - also called score y_{i1}.
- Maximize: Sample variance of the scores y_{i1}.

Second PC

- The second principal component direction is the vector $\beta_2 = (\beta_{12}, \ldots, \beta_{p2})'$ s.t.

$$\max_{\beta_{12},\ldots,\beta_{p2}} \left\{ \frac{1}{n} \left(\sum_{j=1}^{p} \beta_{j2} x_{ij} \right)^2 \right\}$$

$$\text{subject to } \sum_{j=1}^{p} \beta_{j2}^2 = 1, \sum_{j=1}^{p} \beta_{j1}\beta_{j2} = 0$$

- First and second principal components must be orthogonal.
- Scores $(y_{i1}, i = 1, \ldots, n)$ and $(y_{i2}, i = 1, \ldots, n)$ are independent.

Optimization for PCA

- PCA can be performed in two different ways:
- The singular value decomposition of X:

$$X = U\Sigma B^T$$

  where the i-th column of B is the i-th principal component direction β_i.
- The Eigen-Decomposition of $X^T X$:

$$X^T X = B\Sigma^2 B^T$$

Using R

- Two packages `prcomp` and `princomp`.
- Difference:

prcomp: The calculation is done by a singular value decomposition of the (centered and possibly scaled) data matrix, not by using eigen on the covariance matrix. This is generally the preferred method for numerical accuracy.

primcomp: uses Eigen-decomposition.

US Arrest data

Reminiscient of the US Crime data, but different.

Description

This data set contains statistics, in arrests per 100,000 residents for assault, murder, and rape in each of the 50 US states in 1973. Also given is the percent of the population living in urban areas.

Data

```
states=row.names(USArrests)
states
```

```
##  [1] "Alabama"        "Alaska"         "Arizona"        "Arkansas"
##  [5] "California"     "Colorado"       "Connecticut"    "Delaware"
##  [9] "Florida"        "Georgia"        "Hawaii"         "Idaho"
## [13] "Illinois"       "Indiana"        "Iowa"           "Kansas"
## [17] "Kentucky"       "Louisiana"      "Maine"          "Maryland"
## [21] "Massachusetts"  "Michigan"       "Minnesota"      "Mississippi"
## [25] "Missouri"       "Montana"        "Nebraska"       "Nevada"
## [29] "New Hampshire"  "New Jersey"     "New Mexico"     "New York"
## [33] "North Carolina" "North Dakota"   "Ohio"           "Oklahoma"
## [37] "Oregon"         "Pennsylvania"   "Rhode Island"   "South Carolina"
## [41] "South Dakota"   "Tennessee"      "Texas"          "Utah"
## [45] "Vermont"        "Virginia"       "Washington"     "West Virginia"
## [49] "Wisconsin"      "Wyoming"
```

Variable Names

```
names(USArrests)
```

```
## [1] "Murder"   "Assault"  "UrbanPop" "Rape"
```

```
apply(USArrests, 2, mean)
```

```
##   Murder  Assault UrbanPop     Rape
##    7.788  170.760   65.540   21.232
```

```
apply(USArrests, 2, var)
```

```
##     Murder    Assault   UrbanPop       Rape
##   18.97047 6945.16571  209.51878   87.72916
```

- If we failed to scale the variables before performing PCA, then most of the principal components that we observed would be driven by the Assault variable, since it has by far the largest mean and variance.

PCA

```r
pr.out=prcomp(USArrests, scale.=TRUE)
names(pr.out)
```

```
## [1] "sdev"     "rotation" "center"   "scale"    "x"
```

```r
pr.out$center
```

```
##   Murder  Assault UrbanPop     Rape
##    7.788  170.760   65.540   21.232
```

```r
pr.out$scale
```

```
##    Murder   Assault  UrbanPop      Rape
##  4.355510 83.337661 14.474763  9.366385
```

Loadings

The rotation matrix provides the principal component loadings; each column of `pr.out$rotation` contains the corresponding principal component loading vector.

```
pr.out$rotation
```

```
##                 PC1        PC2        PC3         PC4
## Murder   -0.5358995 -0.4181809  0.3412327  0.64922780
## Assault  -0.5831836 -0.1879856  0.2681484 -0.74340748
## UrbanPop -0.2781909  0.8728062  0.3780158  0.13387773
## Rape     -0.5434321  0.1673186 -0.8177779  0.08902432
```

```
dim(pr.out$x)
```

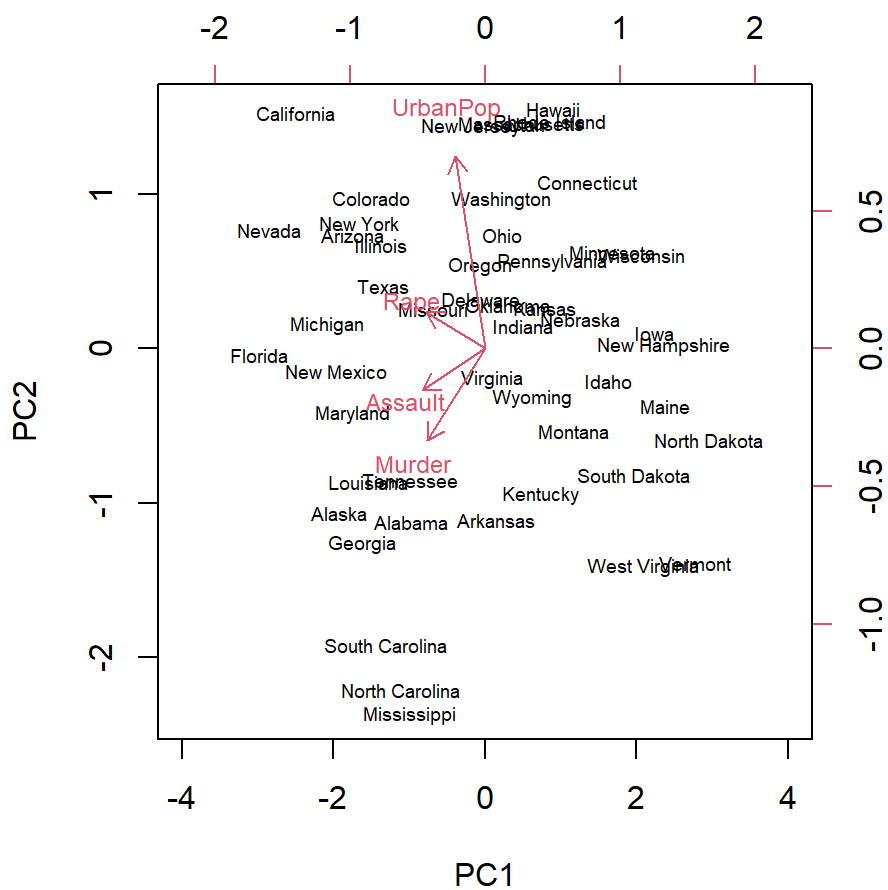
```
## [1] 50  4
```

Biplots

- Biplot provides an efficient way of visualizing together objects (observations) and variables.
- The emphasis of the PCA of the city crime data was on cities (objects).
- However, a graphical representation of the data that contains some information about the variables is particularly useful.

Biplot

```
biplot(pr.out, scale=0,  cex=c(0.6, 0.75),xlim=c(-4,4))
```

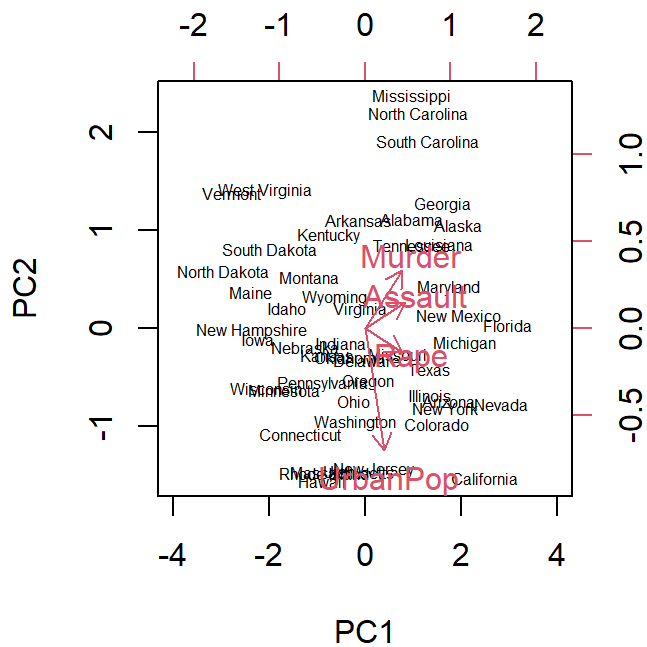


The scale=0 argument to biplot() ensures that the arrows are scaled to represent the loadings.

Biplot

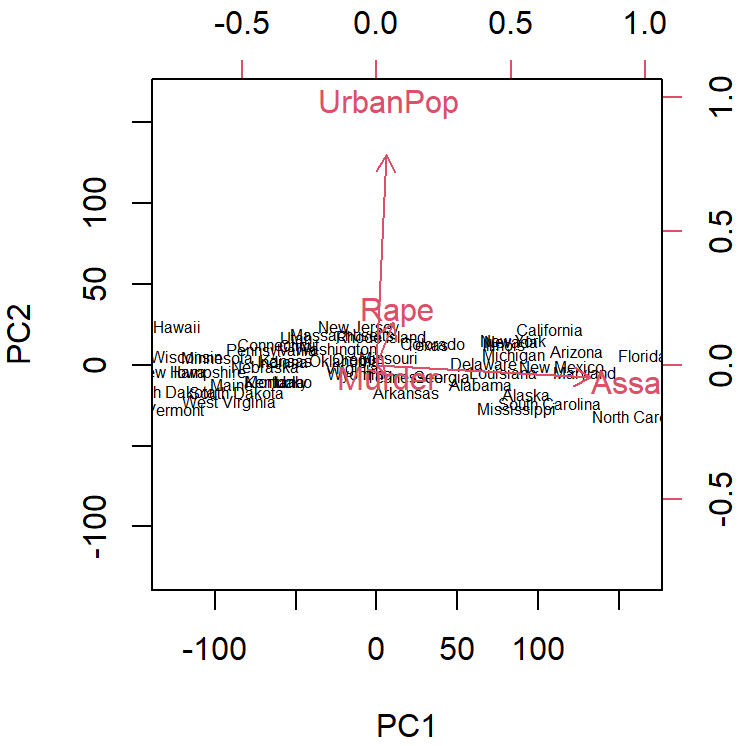
The principal components are only unique up to a sign change

```
pr.out$rotation=-pr.out$rotation; pr.out$x=-pr.out$x
biplot(pr.out, scale=0,cex=c(1/2, 1),xlim=c(-4,4))
```



Importance of Scaling

```
biplot(prcomp(USArrests, scale. = F), scale=0, cex=c(1/2, 1))
```



Standard Deviations

```
pr.out$sdev
```

```
## [1] 1.5748783 0.9948694 0.5971291 0.4164494
```

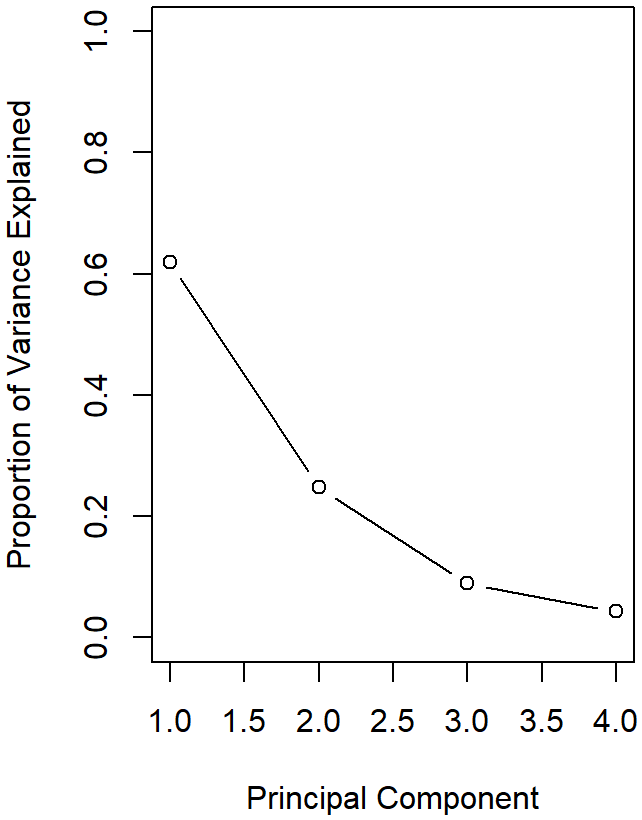
```
pr.var=pr.out$sdev^2
pr.var
```

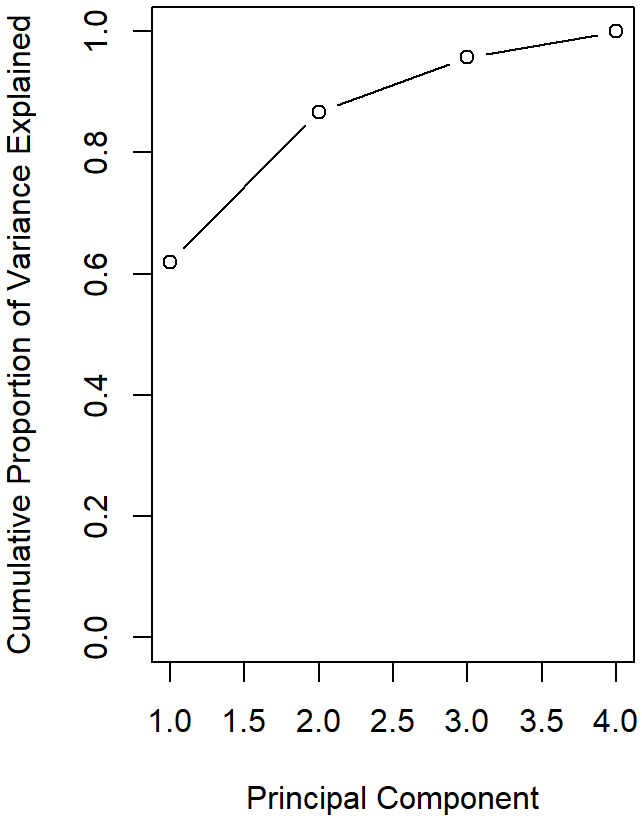
```
## [1] 2.4802416 0.9897652 0.3565632 0.1734301
```

```
pve=pr.var/sum(pr.var)
pve
```

```
## [1] 0.62006039 0.24744129 0.08914080 0.04335752
```

Proportion of Variance Explained





More Examples

- Now we will see two more examples, using the princomp package.
- Recall that the `prcomp` and `princomp` package both performs PCA. One does SVD on the X matrix and the other does Eigen-decomposition on the $X^T X$ matrix.
- They are **almost** identical althouch `prcomp` is preferred by some.

PCA for Crime Data-set

The following R commands perform PCA on the city crime dataset. The data give crime rates per 100,000 people for the 72 largest US cities in 1994.

The variables are: 1) Murder 2) Rape 3) Robbery 4) Assault 5) Burglary 6) Larceny 7) Motor Vehicle Thefts

The Data

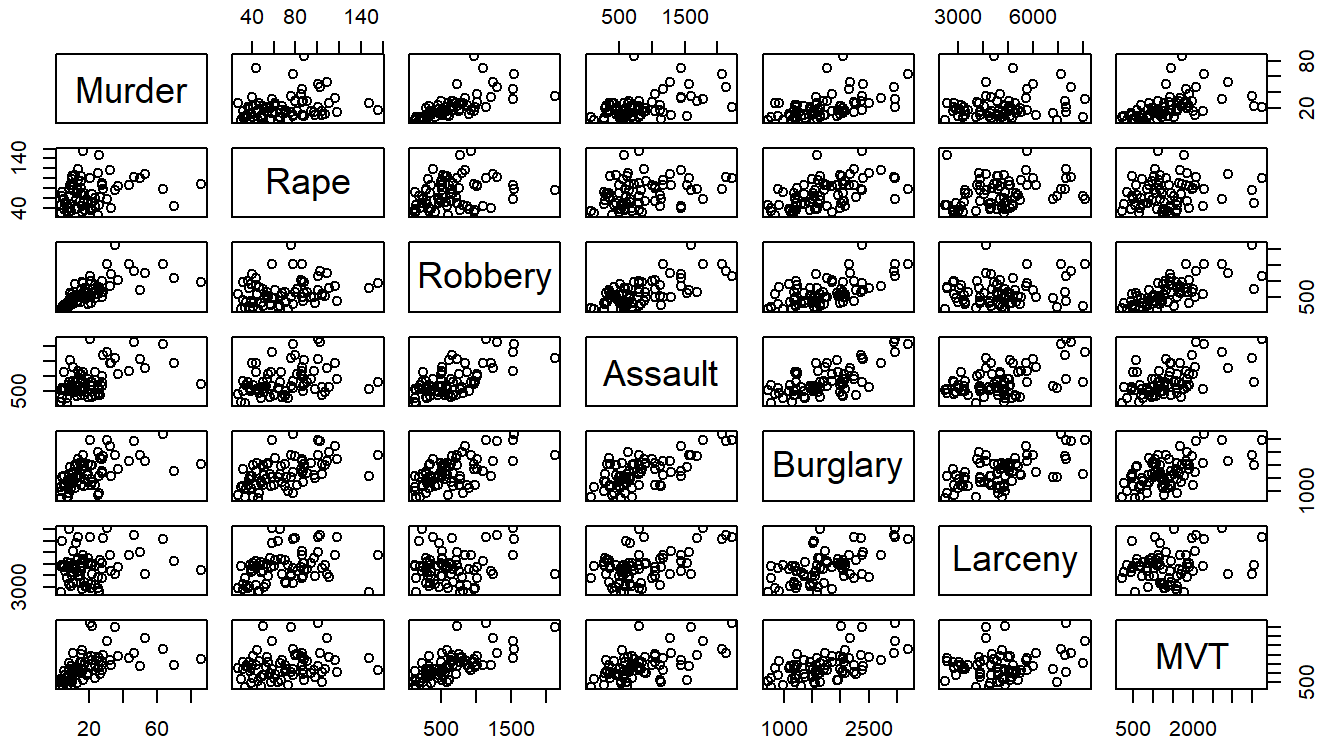
The data are in the “city_crime.txt” file that I read next.

```
setwd("~/GitHub/DattaHub.github.io/data-analytics/data")
crime<-read.table("city_crime.txt")
```

- This is city-wise crime data rather than state-wise like the last example.

Scatterplot Matrix

```
pairs(crime)
```



R

Then apply PCA on the data by rescaling and centering the data using the R function `princomp`:

```
pca.crime<-princomp(scale(crime),cor=FALSE)
```

Look at the results

```
summary(pca.crime)
```

```
## Importance of components:
##                          Comp.1    Comp.2    Comp.3     Comp.4     Comp.5
## Standard deviation     1.934449 1.0873855 0.8710005 0.70936242 0.57288477
## Proportion of Variance 0.542114 0.1712944 0.1099039 0.07289747 0.04754564
## Cumulative Proportion  0.542114 0.7134084 0.8233123 0.89620976 0.94375539
##                           Comp.6     Comp.7
## Standard deviation     0.4585486 0.42187341
## Proportion of Variance 0.0304612 0.02578341
## Cumulative Proportion  0.9742166 1.00000000
```

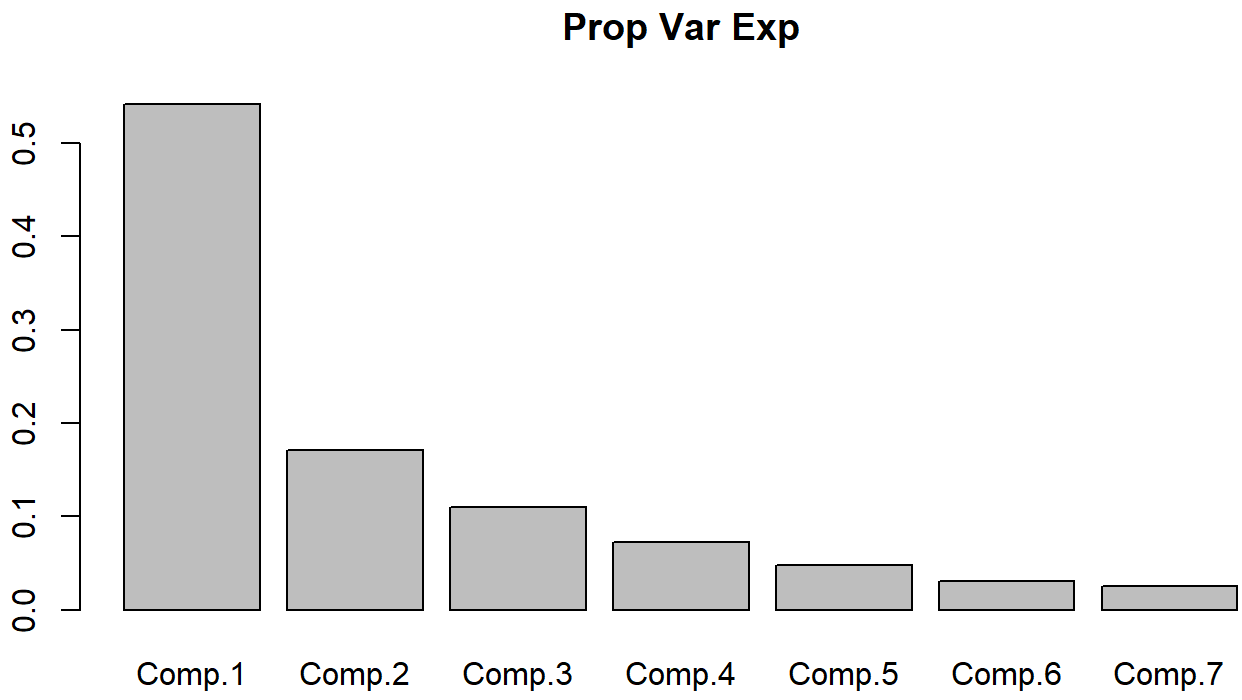
Loadings

```
loadings(pca.crime)
```

```
##
## Loadings:
##          Comp.1 Comp.2 Comp.3 Comp.4 Comp.5 Comp.6 Comp.7
## Murder    0.370  0.339  0.202  0.717  0.277  0.220  0.262
## Rape      0.249 -0.467  0.783 -0.159         0.267
## Robbery   0.426  0.388                -0.194 -0.147 -0.776
## Assault   0.434        -0.282        -0.767  0.118  0.358
## Burglary  0.450 -0.238                 0.242 -0.794  0.220
## Larceny   0.276 -0.606 -0.492  0.209  0.264  0.303 -0.331
## MVT       0.390  0.303 -0.134 -0.644  0.399  0.351  0.204
##
##                Comp.1 Comp.2 Comp.3 Comp.4 Comp.5 Comp.6 Comp.7
## SS loadings     1.000  1.000  1.000  1.000  1.000  1.000  1.000
## Proportion Var  0.143  0.143  0.143  0.143  0.143  0.143  0.143
## Cumulative Var  0.143  0.286  0.429  0.571  0.714  0.857  1.000
```

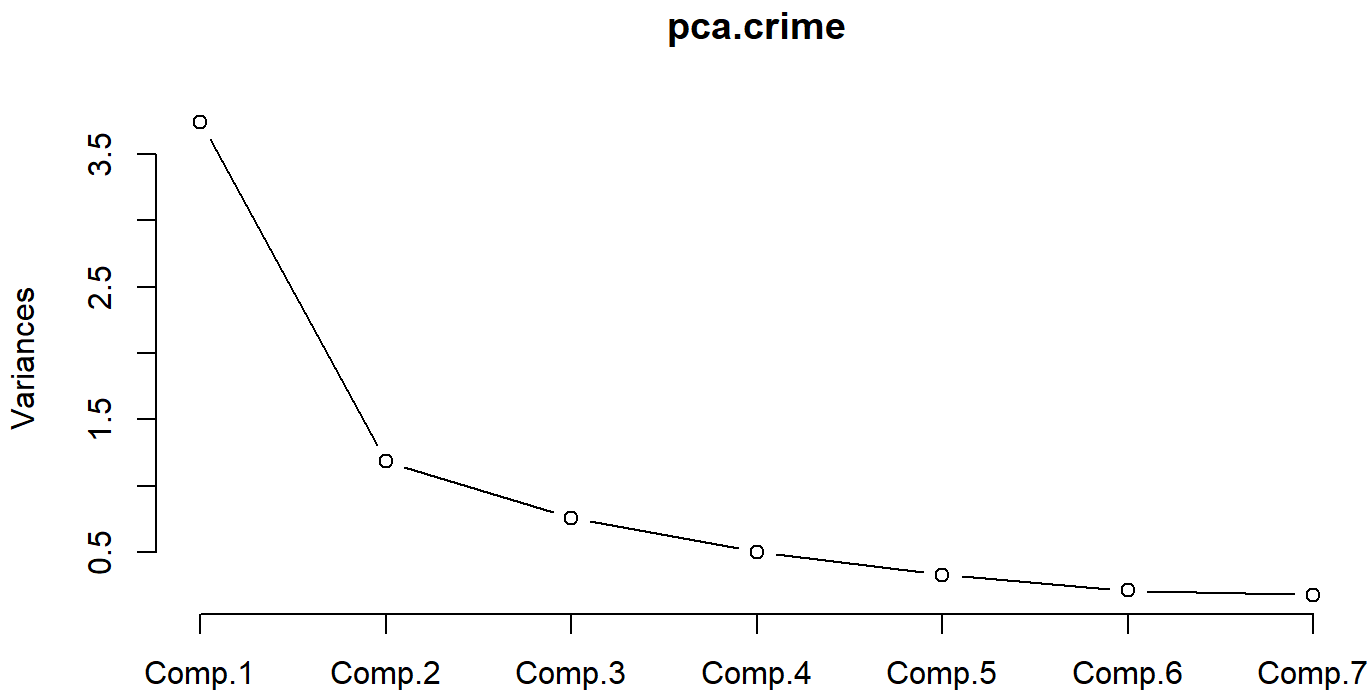
Proportion of Variance

```
PoV <- pca.crime$sdev^2/sum(pca.crime$sdev^2)
barplot(PoV, main="Prop Var Exp")
```



How many?

```
screeplot(pca.crime, type="line")
```



Calculate the principal component scores

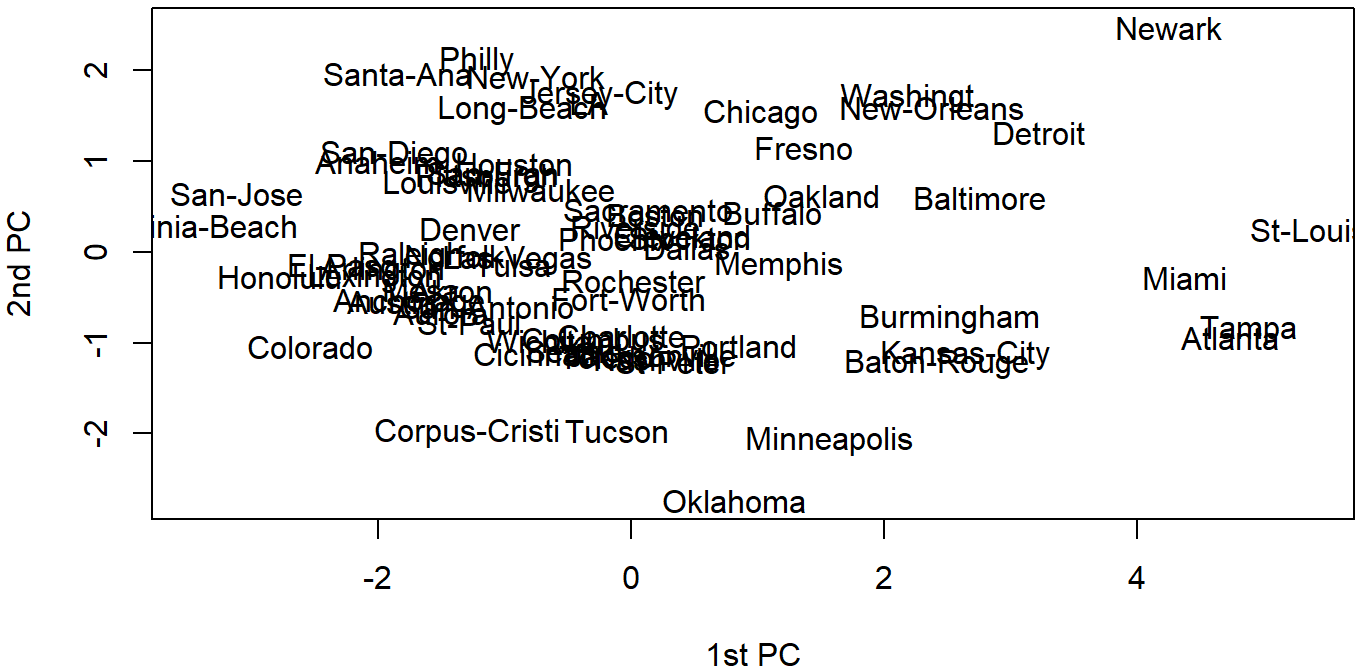
Once we calculate the principal components, we can plot them against each other in order to produce low-dimensional views of the data.

First we calculate the principal component scores.

```
pcs.crime<-predict(pca.crime)
```

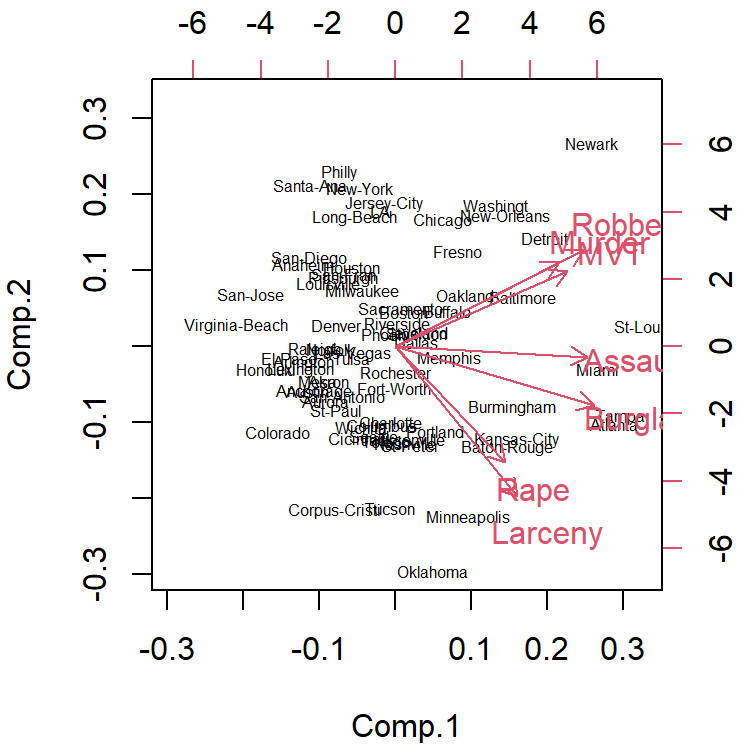
Plot the first 2 PCs.

```
plot(pcs.crime[,1:2],type="n",xlab='1st PC',ylab='2nd PC')
text(pcs.crime[,1:2],row.names(crime))
```



Biplot

```
biplot(pca.crime, cex = c(0.5,1))
```



Questions

1. Which cities are most safe? Which are most crime-prone?
2. How to interpret high/low values in the second PC?

Subtle difference

- For scaling, `prcomp` uses $n-1$ as denominator but `princomp` uses n as its denominator.

```
pc.cr<-prcomp(crime, scale=TRUE, cor=TRUE, scores=TRUE)
pc.cr1<-princomp(crime, scale=TRUE, cor=TRUE, scores=TRUE)
pc.cr$scale
```

```
##     Murder       Rape    Robbery    Assault   Burglary    Larceny        MVT
##   15.49703   28.21087  394.73973  474.47209  562.34922 1345.85980  728.40358
```

```
pc.cr1$scale
```

```
##     Murder       Rape    Robbery    Assault   Burglary    Larceny        MVT
##   15.38904   28.01427  391.98890  471.16562  558.43036 1336.48087  723.32753
```

Iris Data

- We consider the iris dataset again.
- Here the geometric features of the iris flowers can be used to cluster them into different species: setosa, versicolor, virginica.

```
data(iris)
str(iris)
```

```
## 'data.frame':    150 obs. of  5 variables:
##  $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
##  $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
##  $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
##  $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
##  $ Species     : Factor w/ 3 levels "setosa","versicolor",..: 1 1 1 1 1 1 1 1 1 1 ...
```

```
dat.iris = data.matrix(iris[,1:4]) ## features except the species name.
```

Add noise and shuffle columns

- We add 6 extra columns of random normal noise. These extra columns have no information about the species.
- We add the noise columns as columns 1-6, so now we have extra dimensions that we'd like to reduce.

```
set.seed(2023)
dat.iris = cbind(array(rnorm(150*6), c(150, 6)),dat.iris)
colnames(dat.iris)[5:10] <- paste('V', 1:6, sep='')
```

- Now we have original variables as well as noise variables but we don't know which is which.
- Let's see if PCA can still detect the species.

Center and Scale before PCA

```
dat.iris = scale(dat.iris, center=TRUE, scale=TRUE)
```

Perform principal component analysis on correlation matrix

```
pca.iris <- princomp(dat.iris, cor=TRUE)
```

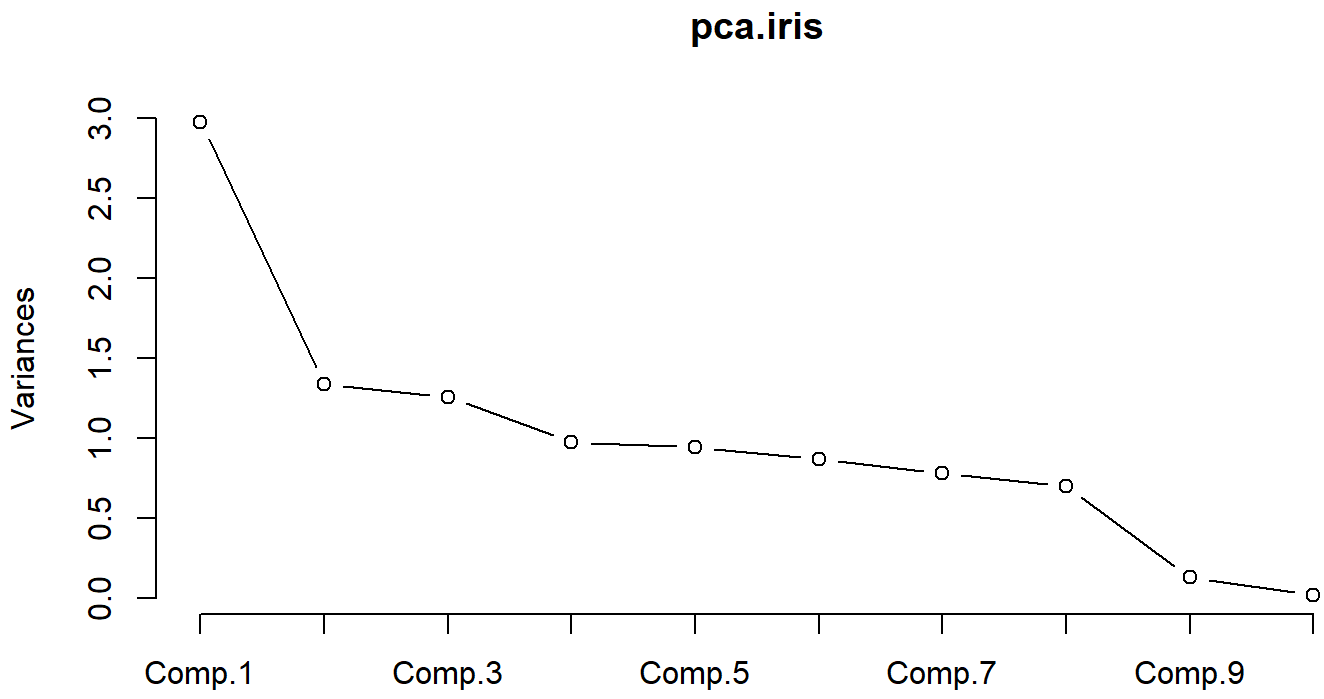
Summarize PCA

```
summary(pca.iris)
```

```
## Importance of components:
##                           Comp.1    Comp.2    Comp.3     Comp.4     Comp.5
## Standard deviation     1.7255297 1.1567711 1.1219589 0.98784253 0.97180302
## Proportion of Variance 0.2977453 0.1338119 0.1258792 0.09758329 0.09444011
## Cumulative Proportion  0.2977453 0.4315572 0.5574364 0.65501966 0.74945977
##                            Comp.6     Comp.7     Comp.8     Comp.9     Comp.10
## Standard deviation     0.93381666 0.88361291 0.83775736 0.36128293 0.142317209
## Proportion of Variance 0.08720136 0.07807718 0.07018374 0.01305254 0.002025419
## Cumulative Proportion  0.83666113 0.91473831 0.98492205 0.99797458 1.000000000
```

Choose no. of PCs to look into

```
screeplot(pca.iris, type="line")
```



Calculate PC scores

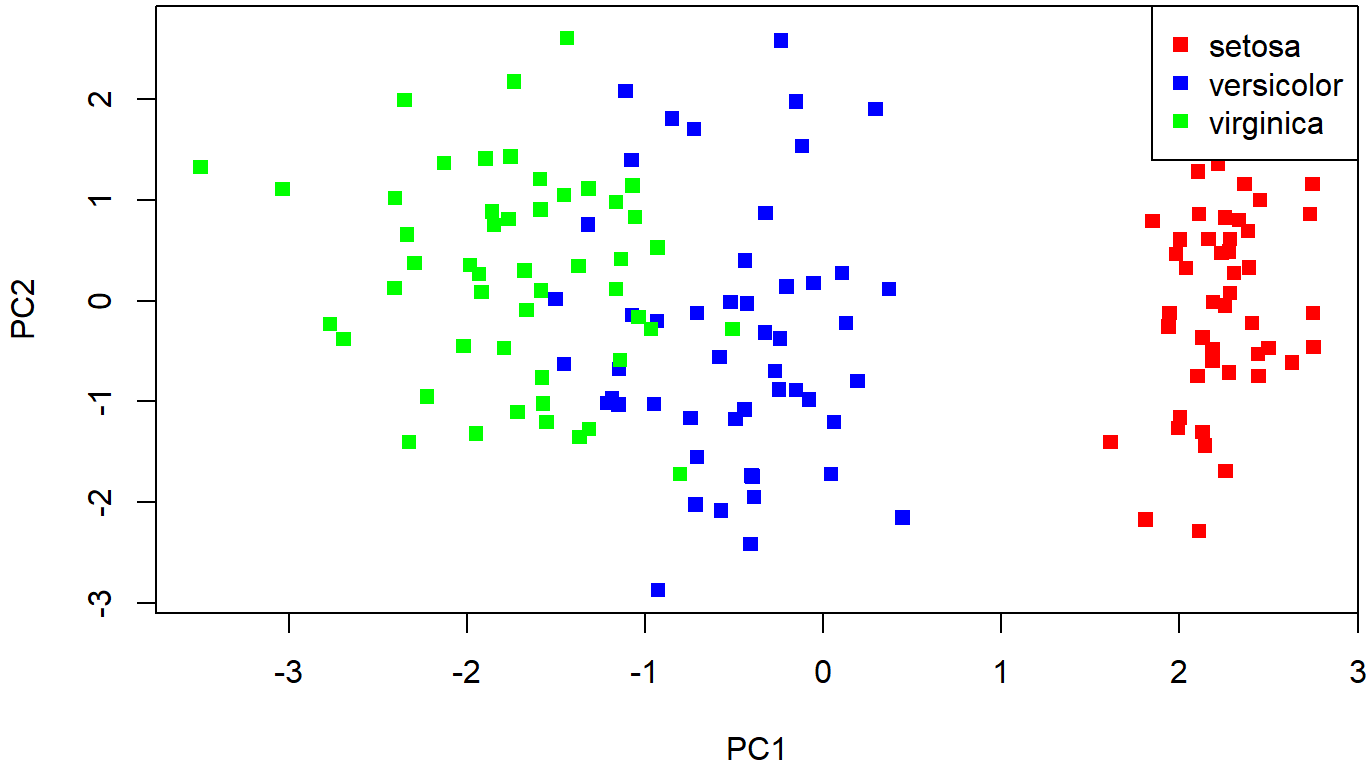
```
pcs.iris <- predict(pca.iris)
```

Set color code for observations from different groups

```
col.code = as.character(iris$Species)
col.code[col.code == "setosa"] = 'red'
col.code[col.code == "versicolor"] = 'blue'
col.code[col.code == "virginica"] = 'green'
```

Plot first two principal components

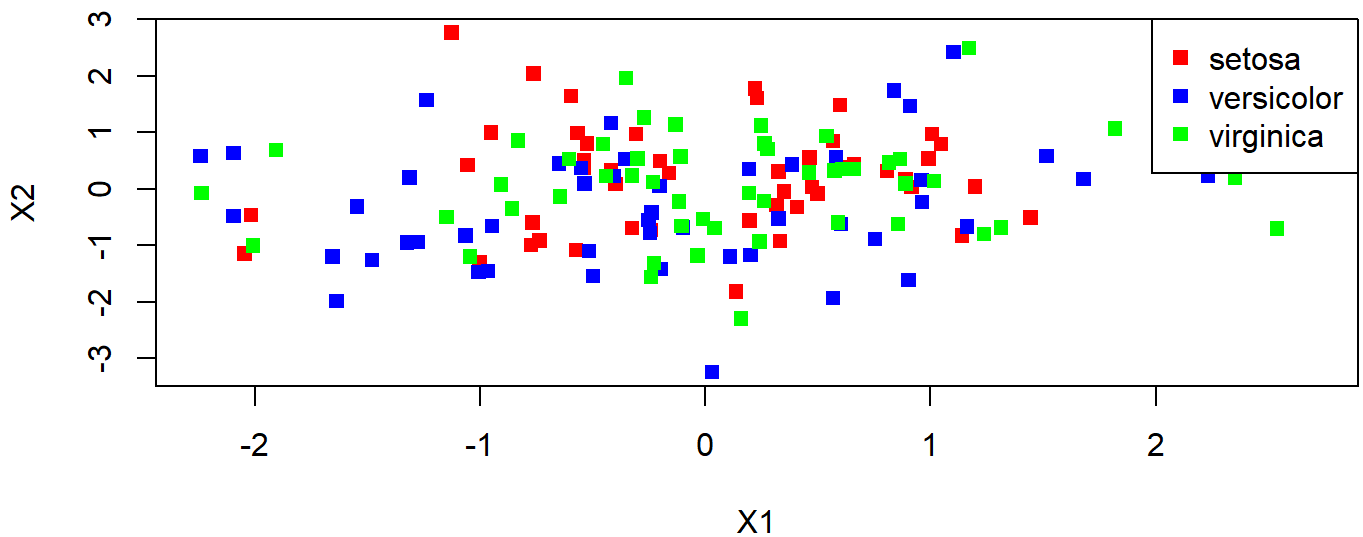
```
plot(pcs.iris[,1:2], type='p', pch=15, col=col.code, xlab="PC1", ylab="PC2")
legend("topright", col=c('red', 'blue', 'green')
     , legend = c('setosa', 'versicolor', 'virginica'), pch = 15
      )
```



Importance of PC

- Plot first two variables $X[, 1:2]$ instead of the first two principal components.
- Do we see any clustering?

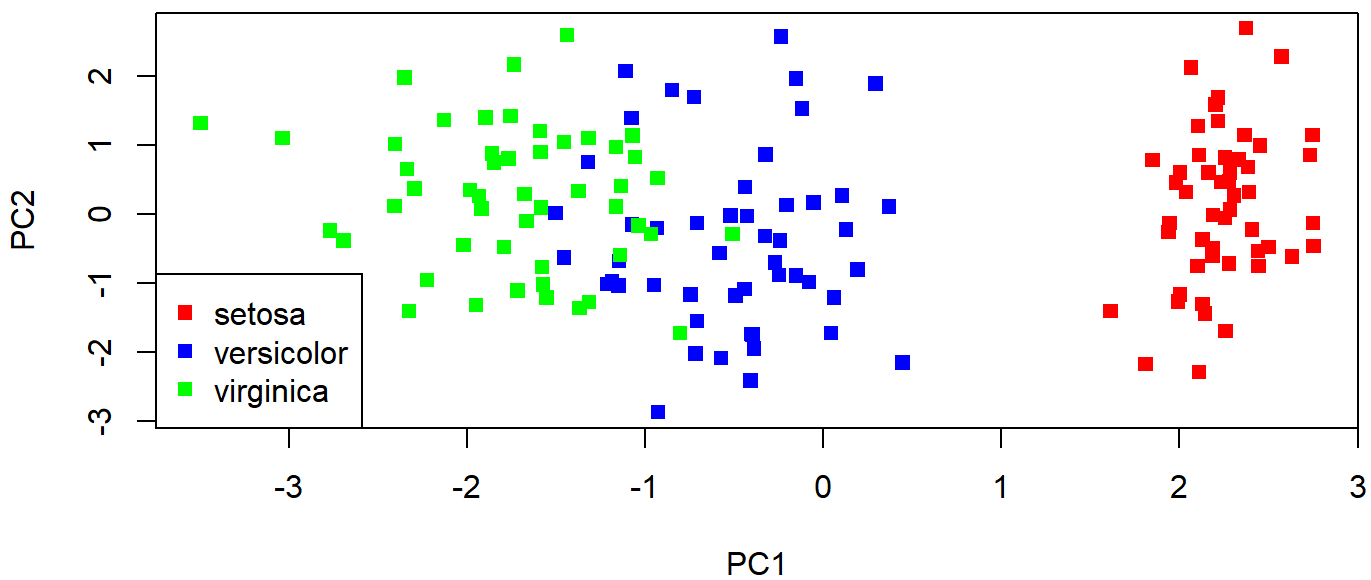
```
plot(dat.iris[,1:2], type='p', pch=15, col=col.code, xlab="X1", ylab="X2")
legend("topright", col=c('red', 'blue', 'green'),legend = c('setosa', 'versicolor', 'virginica'), pch = 15)
```



Importance of PC

When we have many variables, projecting observations into top few PC scores help avoid searching over important variables.

```
plot(pcs.iris[,1:2], type='p', pch=15, col=col.code, xlab="PC1", ylab="PC2")
legend("bottomleft", col=c('red', 'blue', 'green'), legend = c('setosa', 'versicolor', 'virginica')
     , pch = 15
      )
```



Principal components regression

Fix notations

Each of the eigenvectors ϕ_j is called a **principal component**.

It is basically finding the principal axes on which the data vary.

- Axes are given by ϕ_j,
- and variability by d_j^2.

Principal components regression

The most common application of principal components is in creating a low-dimensional set of orthogonal predictors.

We presume that the ϕ_j have been ordered by decreasing d_j^2,

- so that $z_1 = X\phi_1$ has the largest sample variance amongst all normalized linear combinations of the columns of X.

Principal components regression (PCR)

- forms the derived input columns $z_j = X\phi_j$,
- and the regresses y on $z_1, \ldots, z_m$ for some $m \ll p$.

Sum of univariate regressions

On you homework you will show that since the z_j's are orthogonal, the regression is just a sum of univariate regressions.

Ignoring the intercept,

$$\hat{y}^{\text{pcr}} = \sum_{j=1}^{m} \hat{\theta}_j z_j, \quad \text{where} \quad \hat{\theta}_j = z_j^\top y / z_j^\top z_j$$

- although $\hat{\theta} = (Z_{(m)}^\top Z_{(m)})^{-1} Z_{(m)}^\top y$ works too (its just more computationally expensive).

The thinking is that y will tend to vary most in the directions of high variance inputs $z_1, z_2, \ldots$, which may or may not be true.

- We can do `step`-wise variable selection,
- and the greedy solution will be optimal (in a sense, for those PCs).

Backing out the coefficients for X

Since the z_j are each a linear combination of the original x_j, we can express the solution $\hat{\theta}$ in terms of the original (scaled) x_j:

$$\hat{\beta}^{\text{pcr}}(m) = \sum_{j=1}^{m} \hat{\theta}_j v_j.$$

Notice that even though the sum only goes to m,

- implying that $(d - m)$ of the θ_j-coefficients for $j > m$ are effectively zero,
- all of the $\hat{\beta}_j^{\text{pcr}}$ coefficients, $j = 1, \ldots, p$, are nonzero.

Toy PCR example

Consider data on

- scores of six attributes from a sensory panel (potential y-values)
- as predicted by measurements of five psysico-chemical quality parameters
- on $n = 16$ olive oil samples (5 Greek, 5 Italian, 6 Spanish).

Read in the data, focusing on the first attribute (as our y-variable), and standardize the X's.

```
library(pls)
data(oliveoil)
y <- as.numeric(oliveoil$sensory[,1])
X <- as.matrix(oliveoil$chemical)
for(i in 1:ncol(X)) { X[,i] <- X[,i] / sd(X[,i]) }
```

PCA calculations

First the Eigen-decomposition

```
S <- cov(X)
eS <- eigen(S, symmetric=TRUE)
```

Then project onto the PC-space

```
Z <- X %*% eS$vectors
colnames(Z) <- paste("PC", 1:5, sep="")
cumsum(eS$values)/sum(eS$values)
```

```
## [1] 0.5851954 0.8207838 0.9561394 0.9892067 1.0000000
```

- The first three PCs are probably sufficient.

PCA extensions

Using only the first three PCs, add an intercept and calculate $\hat{\theta}$.

```
Zp <- cbind(1, cbind(Z[,1:3]))
theta.pcr <- drop(solve(t(Zp) %*% Zp) %*% t(Zp) %*% y)
theta.pcr
```

```
##                   PC1        PC2        PC3
## 129.103033   7.265572  -2.889501   7.242582
```

Now map $\hat{\theta}$ back into β and X space.

```
beta.pcr <- matrix(c(theta.pcr[1], eS$vectors[,1:3] %*% theta.pcr[-1]), nrow=1)
colnames(beta.pcr) <- c("Intercept", colnames(oliveoil$chemical))
beta.pcr
```

```
##      Intercept   Acidity Peroxide      K232      K270       DK
## [1,]   129.103 -5.406939 -1.34047 -2.905699 -8.539748 1.091108
```

Library routine

The `pls` package provides a routine automating the process.

```
fit.pcr <- pcr(y ~ X, ncomp=3)
rbind(automated=coef(fit.pcr, intercept=TRUE), byhand=as.numeric(beta.pcr))
```

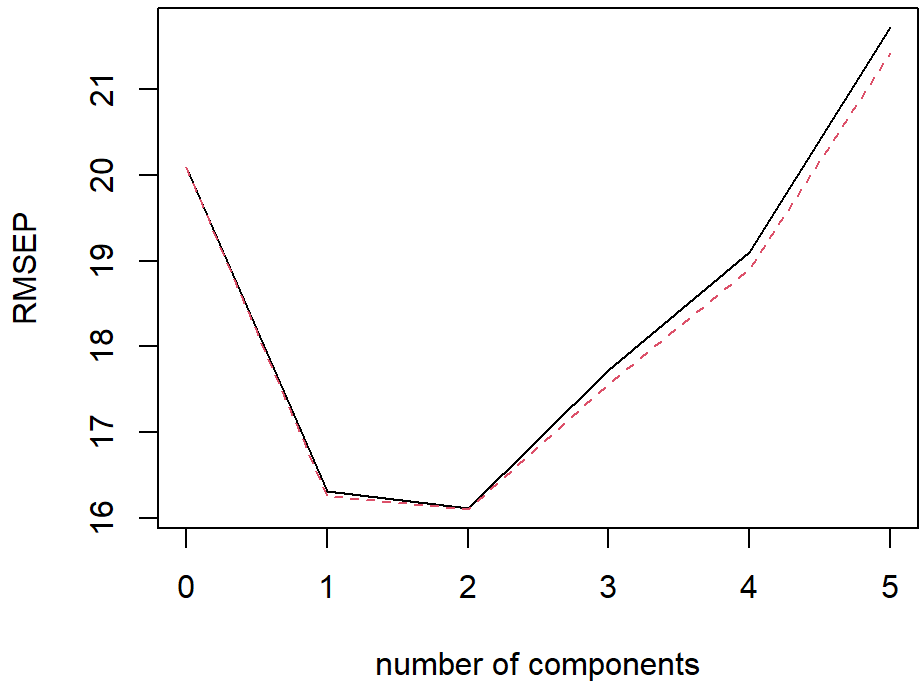
```
##              [,1]      [,2]     [,3]      [,4]      [,5]     [,6]
## automated 129.103 -5.406939 -1.34047 -2.905699 -8.539748 1.091108
## byhand    129.103 -5.406939 -1.34047 -2.905699 -8.539748 1.091108
```

`pls` will also do CV for you.

```
cv.pcr <- pcr(y ~ X, validation="LOO")
RMSEP(cv.pcr)
```

```
##        (Intercept)  1 comps  2 comps  3 comps  4 comps  5 comps
## CV            20.1    16.31    16.11    17.73     19.1    21.73
## adjCV         20.1    16.26    16.10    17.56     18.9    21.42
```

CV visual

Perhaps we can get away with as few as one or two PCs.

```
plot(RMSEP(cv.pcr), main="")
```